

UE25CS243A: Computer Networks — Mini Project Guide

Team 11: Network Time Synchronization System (Project #23)

1. Executive Summary & Project Details

- **Course:** Computer Networks (UE25CS243A), PES University
- **Team:** Team 11 (3 Members)
- **Project Assigned:** Project #23 — Network Time Synchronization System
- **Transport Layer Protocol:** UDP (User Datagram Protocol)
- **Emulation & Control Environment:** Mininet VM + OpenFlow SDN Controller (Ryu / POX) + Open vSwitch (OVS)
- **Total Marks:** 40 Marks (D1: 15 Marks | D2: 25 Marks)

2. Conceptual Overview: What is Network Time Synchronization?

In any distributed computer system, each host node maintains an independent internal clock driven by a physical quartz crystal oscillator. Due to thermal fluctuations, supply voltage changes, and physical manufacturing differences, these hardware oscillators do not tick at identical rates. Over time, clocks experience **clock drift** (gradually running ahead of or behind real wall-clock time).

Why Clock Synchronization is Essential:

- **Distributed System State & Ordering:** Event ordering, message timestamps, and distributed databases (e.g., commit logs, causal ordering) require synchronized nodes.
- **Security & Authentication:** Cryptographic certificates (SSL/TLS), Kerberos tickets, and one-time passwords (TOTP) rely on valid time windows to prevent replay attacks.
- **Log Correlation:** Incident response and network troubleshooting require matching timestamps across multi-tier servers, routers, and firewalls.

Why Use UDP Sockets Instead of TCP?

- **Zero Handshake Latency:** TCP requires a 3-way handshake (SYN, SYN-ACK, ACK) before data transfer, which introduces variable initiation delays.
- **Elimination of Retransmission Jitter:** If a TCP segment is lost or delayed, TCP buffers packets and performs retransmissions (Head-of-Line blocking). This creates unpredictable queuing variations (jitter) that corrupt timestamp calculations.
- **Immediate Wire Transmission:** UDP datagrams are dispatched onto the network wire immediately with minimal protocol overhead, making transmission delay much more deterministic.

3. Protocol Architecture & Mathematical Model

The application implements a 4-timestamp exchange protocol modeled on the Network Time Protocol (NTP):

The 4-Timestamp Exchange:

1. **Step 1 (Client Transmission):** The client reads its local clock at instant **t0**, packages it into a UDP datagram, and sends it to the reference server.
2. **Step 2 (Server Reception):** The server receives the packet and records arrival time **t1** using the reference clock.
3. **Step 3 (Server Transmission):** The server prepares a response containing timestamps (t0, t1), reads its clock at instant **t2** (transmission time), and transmits the UDP reply.
4. **Step 4 (Client Reception):** The client receives the reply datagram and timestamps the instant of arrival as **t3** on its local clock.

Mathematical Formulations:

- **Round-Trip Delay (RTT):**
Delay = (t3 - t0) - (t2 - t1)
Explanation: Measures the pure network transit time in both directions, subtracting the server's internal processing turnaround time (t2 - t1).
- **Clock Offset (Theta):**
Offset = ((t1 - t0) + (t2 - t3)) / 2
Explanation: Quantifies the time difference between the client clock and the reference server clock under the standard assumption of symmetric network transit delay.
* If Offset > 0: The client's clock is behind the server (needs to advance forward).
* If Offset < 0: The client's clock is ahead of the server (needs to slow down or step backward).
- **Client Clock Correction:**
Corrected_Time = Local_Virtual_Clock + Offset

4. Software-Defined Networking (SDN) Integration

In a conventional best-effort network, time-synchronization packets compete for switch buffers with bulky background traffic (file downloads, video streams). When buffer queues fill up (bufferbloat), synchronization packets suffer high latency and severe jitter, causing calculated clock offsets to deviate widely.

How SDN Solves This Problem:

- **Microflow Classification:** The SDN controller instructs OpenFlow switches to inspect packet headers. Time-sync traffic is distinctly classified by its UDP destination port (e.g., port 12300).
- **Strict Queue Prioritization (QoS):** Open vSwitch queues are configured such that UDP port 12300 traffic enters a high-priority queue, while normal background traffic (e.g., port 5001 iperf flows) enters standard or rate-limited queues.
- **Dynamic Path Rerouting (Challenging Scenario):** In a multi-path topology:
 - * **Path A (Primary):** Direct path, carrying default traffic.

- * **Path B (Alternate):** Dedicated or low-latency path.
- * When the SDN controller detects congestion or heavy queuing on Path A, it dynamically pushes OpenFlow flow modification rules (FlowMod) to redirect UDP port 12300 traffic over Path B, keeping time-sync delay and jitter near zero.

5. Team Roles & Work Breakdown (3 Members)

Member 1: Socket Application & Protocol Lead

- **Domain:** Low-Level Socket Programming, Concurrency, and Protocol Engine
- **Deliverables:**
 - `protocol.py`: Packet serialization/deserialization (binary `struct` or JSON), timestamp extraction, and RTT/offset computation formulas.
 - `server.py`: UDP reference time server handling concurrent client requests with non-blocking sockets or threading.
 - `client.py`: Multi-client node simulator with controllable artificial clock drift (e.g., +20 ms per second) and periodic synchronization adjustment loop.
 - Application-level logging: Record (t0, t1, t2, t3, RTT, Offset) to structured CSV files for analysis.

Member 2: SDN Controller & Mininet Topology Lead

- **Domain:** Network Virtualization, OpenFlow Control Plane, and Traffic Management
- **Deliverables:**
 - `custom_topo.py`: Mininet custom Python topology creating multiple switches, redundant paths, client hosts, the reference server, and background load nodes.
 - `controller.py`: OpenFlow 1.3 Ryu/POX controller implementing flow tables and reactive/proactive rule installations.
 - **Dynamic SDN Mechanisms:** Classify UDP port 12300, attach QoS queues, and implement dynamic failover/rerouting when link congestion or artificial delay occurs.
 - Link condition injection: Scripts using Linux `tc` (traffic control) / NetEm to inject controlled delay (e.g., 50 ms) and jitter on specific switch ports.

6. Repository File Structure

- ### Member 3: Integration, Benchmarking & Documentation Lead
- **Domain:** System Orchestration, Performance Benchmarking, and Reporting
 - **Deliverables:**
 - `run_demo.sh` & `benchmark.py`: End-to-end test runner that starts the SDN controller, initializes Mininet, spawns background traffic (iperf), and triggers client sync cycles.
 - Comparative Performance Evaluation: Run tests across two core scenarios:
 1. Baseline Mode: Best-effort network without SDN QoS/rerouting under background load.
 2. SDN-Optimized Mode: Dynamic SDN prioritization/rerouting active under identical load.

- plot_results.py: Python script generating graphs comparing:
 - * Clock Offset Error vs. Time (Baseline vs. SDN)
 - * Round-Trip Delay (RTT) Distribution
 - * Jitter and Convergence Time
- Final documentation: Comprehensive GitHub README, project architecture report, and presentation slides for the viva.

None

```

team11-network-time-sync/
├── README.md                    # Quickstart guide,
architecture overview, and team info
├── requirements.txt             # Dependencies (ryu,
matplotlib, pandas)
|
├── app/                        # Low-level socket programming
code
|   ├── __init__.py
|   ├── protocol.py            # Packet schema, timestamp
encoding, offset math
|   ├── server.py              # High-precision UDP time
server
|   ├── client.py              # Client node with drifted
virtual clock
|   └── background_traffic.py    # Background UDP/TCP load
generator (iperf wrapper)
|
├── sdn/                        # SDN control plane logic
|   ├── __init__.py
|   ├── controller.py          # Ryu OpenFlow controller (QoS
& dynamic rerouting)
|   └── flow_rules.json         # Pre-configured flow rules and
port policies
|
├── topology/                  # Mininet topology definitions
|   ├── __init__.py
|   └── custom_topo.py          # Multi-path network topology
definition
|

```

```

├─ evaluation/                                # Performance evaluation &
benchmarking
|   └─ benchmark.py                          # Automated test harness across
multiple conditions
|   └─ plot_results.py                      # Generates comparative charts
|   └─ data/                               # Raw CSV logs of measured
offsets and delays
|       └─ plots/                          # Rendered graphs (Offset vs
Time, Jitter comparisons)
|
├─ scripts/                                # Operational shell scripts
|   └─ setup_env.sh                        # Installs Mininet, OVS, and
Python packages
|   └─ run_controller.sh                   # Starts the Ryu controller
|       └─ run_demo.sh                    # Automated single-command full
system demo
|
└─ docs/                                  # Project documentation &
presentation
    └─ architecture_diagram.png           # Complete system architecture
schematic
        └─ D1_Design_Document.pdf         # Milestone 1 submission report
        └─ D2_Final_Report.pdf            # Milestone 2 final report &
viva notes

```

7. Evaluation Rubrics & Milestone Checklist

Milestone	Rubric Component	Marks	Focus & Implementation Goal
D1 (15 M)	Problem Understanding & Architecture	2	Well-defined objectives, NTP mathematical model, and clear architecture diagram.
	TCP/UDP Socket Implementation	4	Low-level UDP socket client/server with 4-timestamp exchange and concurrency support.
	Mininet + SDN Implementation	4	Custom Mininet multi-path topology and functional OpenFlow switch control.

Milestone	Rubric Component	Marks	Focus & Implementation Goal
D2 (25 M)	Socket–SDN Integration	3	Verification of application UDP packets traversing the SDN switches.
	Initial Testing & Demo	2	End-to-end functionality working under normal, uncongested conditions.
	Complete Application Functionality	4	Multiple client nodes, clock drift simulation, and automatic periodic clock synchronization.
	Dynamic SDN Functionality	6	Dynamic flow classification on UDP port 12300, QoS prioritization, and dynamic path rerouting.
	Challenging Scenario / Robustness	4	Resilient performance under heavy background congestion (iperf) and injected link delay (tc netem).
	Performance Evaluation & Comparison	5	Quantitative comparison and plots of Synchronization Error: Baseline vs. SDN-Optimized.
	Documentation, Final Demo & Viva	6	Clean GitHub repository, reproducible demo script (run_demo.sh), and individual viva readiness.
Total		40	